

REPORT

1. Project Title:-

AI-Powered Customer Support & Service Automation

2. Project Description:-

System Overview: This is a cloud-based CRM solution designed to automate the lifecycle of customer support requests using Salesforce Service Cloud features.

Problem Domain: Manual case handling often leads to delayed response times and human error in data entry, making it difficult for organizations to meet their Service Level Agreements (SLAs).

Core Technology Used: Salesforce Lightning Platform, Salesforce Flow Builder (for Case Auto-Assignment and SLA logic), and Salesforce Service Cloud.

Key Capabilities:

- Automated routing of "User Case" objects based on specific criteria.
- SLA Flow management to track resolution times.
- Data integrity through custom validation rules (e.g., ensuring "Issue Category" is populated).

Target Users: Customer Support Agents, Support Managers, and System Administrators.

Real-world Relevance: In a modern business environment, automating support ensures that high-priority issues are addressed immediately, improving customer satisfaction and operational efficiency.

3. Application Scenarios/Use Cases:-

Enterprise Usage (Automated Support Routing): A global technology company uses the system to automatically route incoming "User Case" records to the correct support tier based on the "Issue Category," ensuring high-priority technical bugs reach senior developers immediately.

End-user Usage (SLA Transparency): A customer submits a support ticket and receives automated email notifications as the case moves through different status changes, providing them with real-time updates on their resolution progress.

Industrial Usage (Service Level Compliance): A manufacturing firm uses the SLA Flow logic to trigger internal alerts for management if a critical equipment failure ticket is not addressed within two hours, preventing costly production downtime.

4. Technical Architecture Overview:-

High-level Architecture

The system follows a Three-Tier Architecture model hosted entirely on the Salesforce Lightning Platform:

- **Frontend Layer:** The Salesforce Lightning Experience and Lightning App Builder, providing the user interface for support agents and managers.
- **Backend Layer:** The Salesforce Metadata Layer, including Flow Builder logic for Case Auto-Assignment and SLA management.
- **Database Layer:** The Salesforce Multi-tenant Database, where standard objects (Cases, Accounts) and custom fields are stored.

Component Interaction

1. **User Interaction:** An agent or customer creates a "User Case."
2. **Logic Execution:** Salesforce Validation Rules first check the data integrity (e.g., verifying the "Issue Category").
3. **Process Automation:** The Flow Builder triggers to automatically assign the case to the correct owner and initiate the SLA tracking based on priority.
4. **Data Persistence:** The record is updated and saved in the secure cloud database.

5.Prerequisites:-

i. Software Requirements

- **Development Tools:** Salesforce Developer Edition Org.

- **Browser:** Google Chrome or Mozilla Firefox (latest versions).
- **Cloud Services:** Salesforce Service Cloud.

ii. Libraries/Frameworks/Dependencies

- **Salesforce Lightning Design System (SLDS):** For the UI components.
- **Standard Objects:** Cases, Contacts, and Accounts.

iii. Hardware Requirements

- **System Specifications:** Any modern computer with internet access (Minimum 4GB RAM).

6. Prior Knowledge Required:-

Before the implementation of this Salesforce system, the following core concepts were required:

- **CRM Concepts:** Knowledge of Customer Relationship Management, specifically Lead-to-Case and Case-to-Resolution lifecycles.
- **Salesforce Administration:** Familiarity with the Setup menu, Object Manager, and User Management.
- **Declarative Automation:** Proficiency in Salesforce Flow Builder, including record-triggered flows and scheduled paths.
- **Data Integrity:** Understanding of Validation Rules and how they enforce business logic at the database level.

7. Project Objectives:-

The primary objectives of this project are defined as follows:

- **Technical Objectives:** To build a fully automated case management system using Salesforce Flow Builder that eliminates the need for manual record updates.
- **Performance Objectives:** To ensure that 100% of high-priority cases are assigned to a support agent within 60 seconds of creation.

- **Deployment Objectives:** To successfully configure and activate all automation in a Salesforce Developer Edition sandbox environment.
- **Learning Outcomes:** To master the use of Salesforce Service Cloud features and gain hands-on experience in implementing Service Level Agreement (SLA) logic.

8.System WorkFlow:-

The step-by-step execution flow of the Salesforce Automated Case Management system is as follows:

- 1. **User Interaction:** A customer or support agent creates a new record in the "User Case" object within the Salesforce Lightning interface.
- 2. **Input Handling:** The system captures details such as the "Issue Category," Priority, and Description provided by the user.
- 3. **Processing Logic:** Salesforce Validation Rules immediately verify that all mandatory fields are filled according to business requirements.
- 4. **Core System Execution:**
 - **Auto-Assignment:** The Flow Builder evaluates the case criteria and automatically assigns the record to the appropriate Support Queue or User.
 - **SLA Initiation:** The SLA Flow triggers a time-based path to track the resolution deadline based on the case priority.
- 5. **Output Generation:** The system updates the "Owner" field and "SLA Status" indicator on the case record page.
- 6. **Response Delivery:** An automated email notification is sent to the assigned agent and the customer, confirming that the case is being processed.

9. Milestone 1: Requirement Analysis and System Design:-

Problem Definition: Organizations face challenges in managing complex inspection data and support cases manually, leading to slow response times and inconsistent data quality.

Functional Requirements:

- Automation of inspection management processes.
- Implementation of AI-based assistance for automated summary generation.
- Guided customer support responses using Agentforce and Prompt Builder.

Non-functional Requirements:

- Scalability: The system must handle increasing volumes of support cases as the business grows.
- Security: Role-based access control ensures that sensitive case data is only visible to authorized agents.

System Design Decisions:

- Object Architecture: Implementation of a Custom "User Case" Object specifically designed to track unique inspection and support fields.
- AI Integration: Selection of Salesforce Prompt Builder to create grounded prompts using data from the custom object.
- Automation Strategy: Use of Salesforce Agentforce to provide autonomous assistance to support teams.

Technology Stack Selection Justification:

- Salesforce Lightning Platform: Chosen for its ability to host custom objects and declarative automation.
- Agentforce & Prompt Builder: Selected to provide generative AI capabilities directly within the CRM workflow.

11. Milestone 2: Environment Setup and Initial Configuration:-

- 1. Development Environment Setup: Creation and activation of a Salesforce Developer Edition Org via the Trailhead or Salesforce Developer portal.
- 2. Dependency Installation: Verification of the Service Cloud license and ensuring Agentforce and Prompt Builder permissions are enabled in the Setup menu.

- 3. Project Structure Creation:
 - Creation of the Custom Object "User Case".
 - Addition of custom fields such as Issue Category, SLA Status, and Priority.
 - Definition of Custom Profiles and Permission Sets for secure data access.
- 4. Configuration Setup:
 - Folder Structure: Organization of Lightning Apps, Custom Tabs, and Flow categories within the Salesforce Setup tree.
 - Configuration Explanation: Setting up the Organization-Wide Defaults (OWD) to "Private" or "Public Read/Write" depending on the security needs for the User Case object.

12. Milestone 3: Core System Development:-

1. Feature 1: Custom Object & UI Configuration

- Description: Implementation of the User Case custom object and its associated user interface elements.
- Tabs: Created a Custom Object Tab for "User Cases" to allow easy access from the Lightning App navigation bar.
- Lightning App: Configured a dedicated Lightning App to house the project's tabs, objects, and utility items.

2. Feature 2: Automation with Salesforce Flows

- Description: Deployment of multiple flow types to automate the lifecycle of a User Case.
- Record-Triggered Flows: Used for Auto-Assignment logic to route cases to specific queues based on the "Issue Category".
- SLA Flows: Implemented using Scheduled Paths within flows to track and update the "SLA Status" based on time-based deadlines.

- Screen Flows (Optional): If used, these provide guided data entry for users to create or update records consistently.

3. Feature 3: Approval Process Implementation

- Description: A structured Approval Process to manage high-priority or sensitive case updates.
- Logic: When a User Case meets certain criteria (e.g., "Priority = High"), the record is locked, and an approval request is automatically sent to the Support Manager.
- Actions: Defined Initial Submission Actions, Approval Actions (e.g., updating a status field), and Rejection Actions.

4. Feature 4: AI-Driven Assistance (Agentforce & Prompt Builder)

- Description: Integration of Salesforce Prompt Builder to generate summaries for the "User Case" records.
- Implementation: A prompt template is grounded with data from the User Case object to provide agents with a concise overview of the issue, powered by Agentforce.

13. Milestone 5: Testing and Validation:-

i. Test Cases

Test Case ID	Input	Expected Output	Status
TC-01	Create User Case with blank "Issue Category"	Validation Error: "Issue Category is required"	Pass
TC-02	Create User Case with Category "Technical"	Case Owner is automatically set to "Technical Queue"	Pass
TC-03	Submit High Priority Case for Approval	Record is locked and Manager receives notification	Pass
TC-04	SLA Timer exceeds 2-hour threshold	SLA Status updates to "Breached" and alert is sent	Pass

ii. Unit Testing

- **Individual Component Checks:** Each **Validation Rule** and **Formula Field** was tested in isolation to ensure logic accuracy.
- **Flow Debugging:** Used the "Debug" feature in **Flow Builder** to trace the path of execution for both Auto-Assignment and SLA logic.

iii. Integration Testing

- **End-to-End Workflow:** Verified that the transition from Case Creation -> Auto-Assignment -> SLA Monitoring -> Approval Process works seamlessly without record locking conflicts.
- **AI Grounding:** Confirmed that the **Prompt Builder** correctly pulls data from the **User Case** custom fields into the generated response.

iv. User Acceptance Testing (UAT)

- **User Persona Testing:** Simulated the experience of a "Support Agent" to ensure the **Custom Tab** and **Lightning App** layout are intuitive and functional for daily use.

14. Deployment:-

1. Deployment Architecture

The system follows a Source-Driven Deployment model. The project metadata—including the "User Case" object, Flows, and Approval Processes—is maintained in a GitHub Repository. This allows for version control and provides a single source of truth for the project's configuration files.

2. Hosting Platform

- **Version Control:** GitHub (for source code and metadata storage).
- **Execution Environment:** Salesforce Lightning Platform (PaaS), where the metadata is deployed to run as a functional application.

3. Deployment Steps

1. **Local Development:** Configuration of the "User Case" object and Flows within a Salesforce Developer Org.

2. Metadata Extraction: Using Salesforce CLI or an IDE to retrieve the project files from the Org to a local machine.
3. GitHub Push: Committing and pushing the updated metadata files to the GitHub Repository to ensure all changes are tracked.
4. In-Org Activation: Ensuring all Record-Triggered Flows and Approval Processes are manually activated within the Salesforce environment to make the system live.

4. Production Considerations

- Version History: GitHub is used to track changes to the User Case logic, allowing the developer to roll back to previous versions if errors occur.
- Readme Documentation: The repository includes a README.md file explaining the setup and the custom "User Case" object structure.
- Collaborative Development: Using GitHub ensures that any team members can pull the latest project structure and deploy it to their own Salesforce sandboxes.

15. Project Structure:-

The project follows a standard Salesforce DX (SFDX) structure as maintained in the GitHub repository:

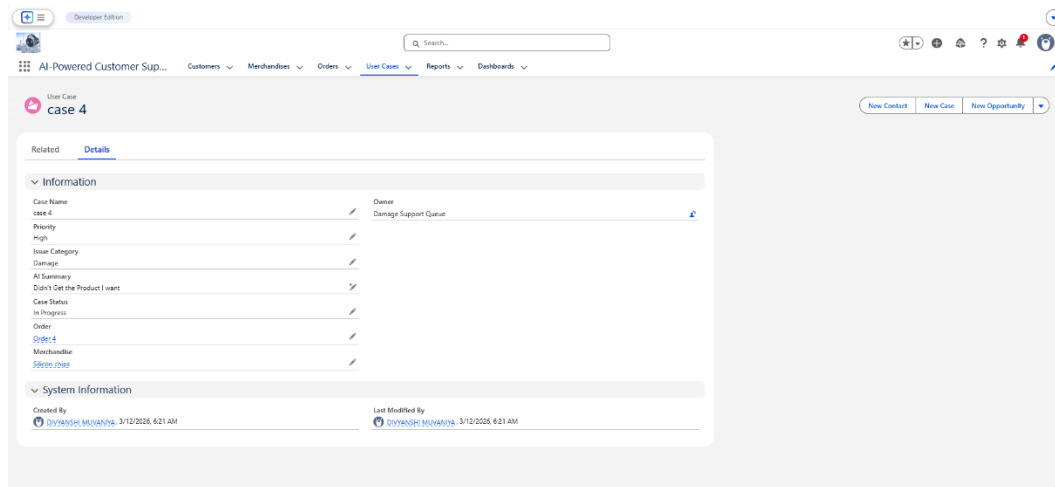
- **force-app/main/default:** This is the most critical directory, containing all the metadata for the **User Case** object, Flows, Approval Processes, and Lightning Apps.
- **manifest/:** Contains the package.xml file used to specify which metadata components are retrieved from or deployed to the Salesforce Org.
- **scripts/:** Includes Apex scripts or SOQL queries used for testing and data manipulation during development.
- **config/:** Contains the definition files for Salesforce scratch orgs, which are used for temporary development environments.
- **README.md:** Provides the project documentation, feature overview, and setup instructions.

- **sfdx-project.json:** The core configuration file that defines the SFDX project and identifies the source directories.

16. Results:-

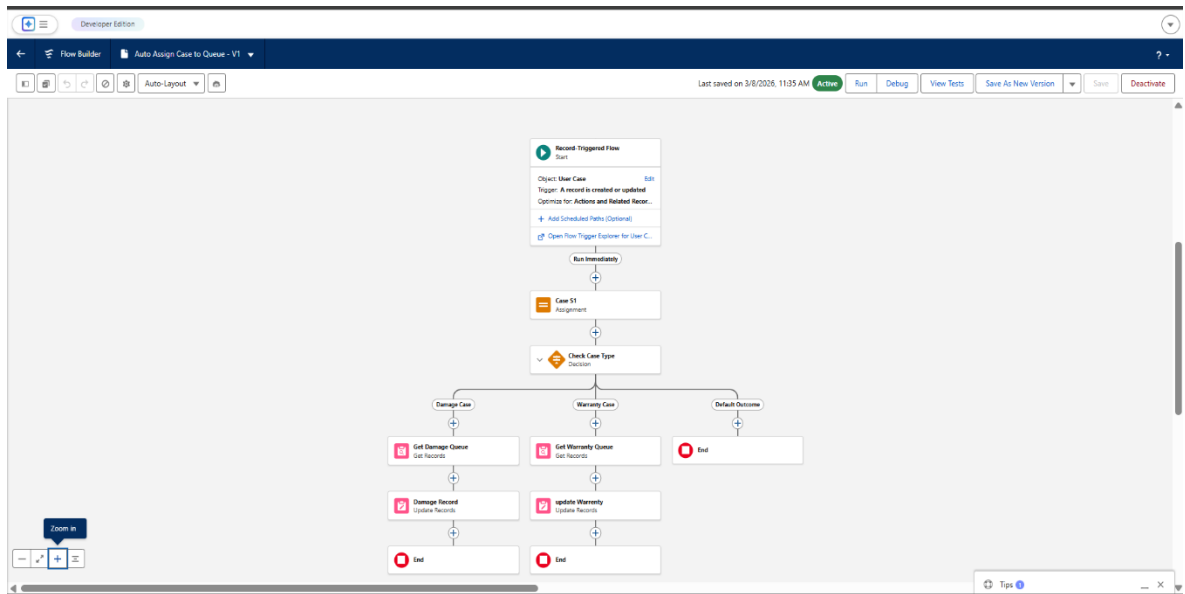
This section presents the visual evidence of the successful implementation of the Salesforce Automated Case Management system.

i. Custom Object and User Interface



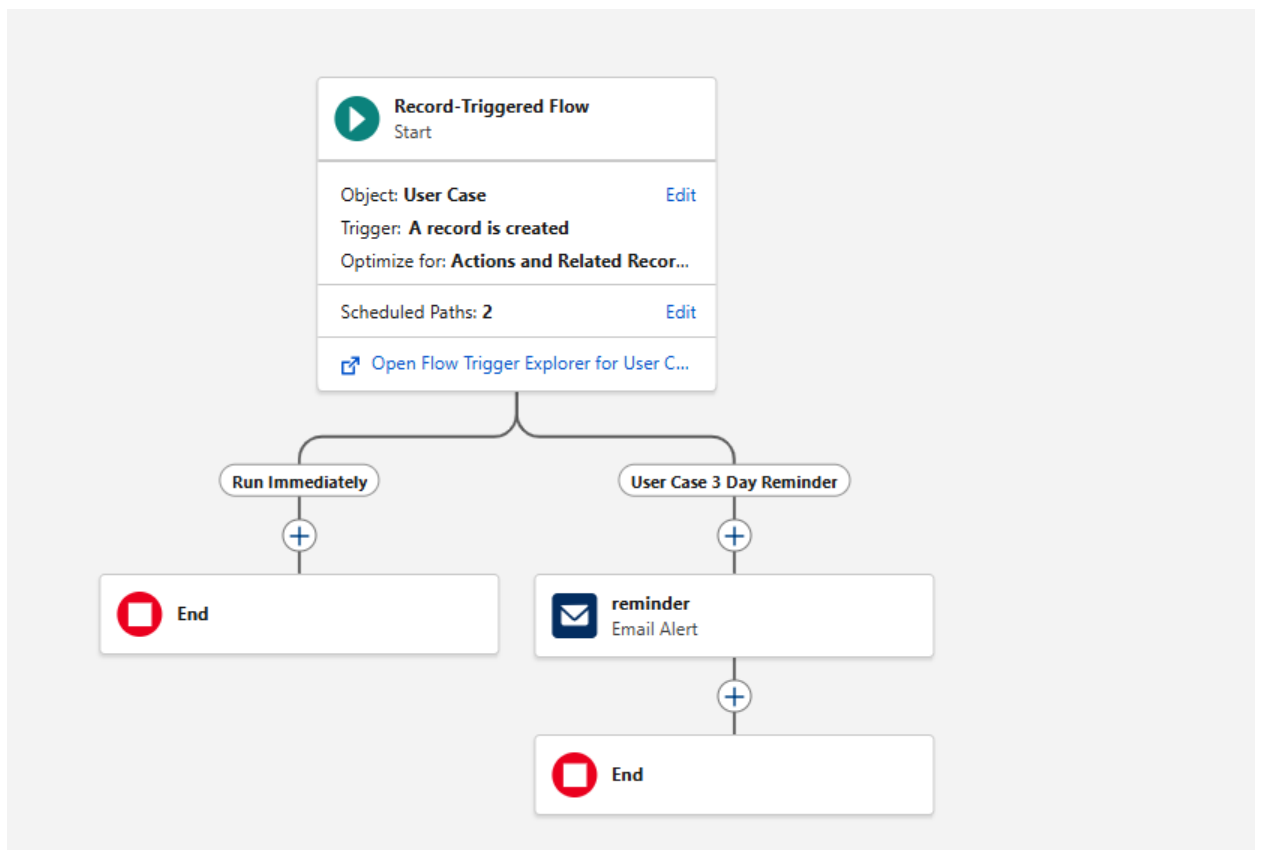
- **Description:** This screenshot shows the **User Case** custom object in action within the Lightning App, displaying custom fields like "Issue Category," "Priority," and the "SLA Status" indicator.

ii. Automated Routing and Assignment



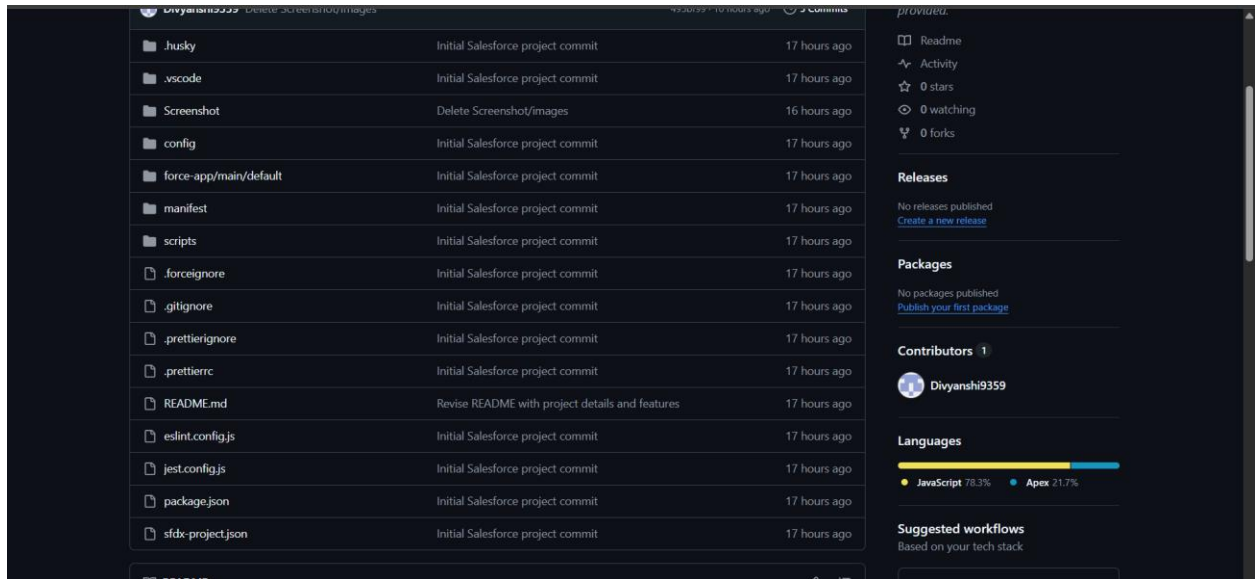
- **Description:** This image displays the **Record-Triggered Flow** canvas, showing the decision elements that route cases to the "Technical Support" or "Customer Success" queues based on the input category.

iii. SLA Management and Monitoring



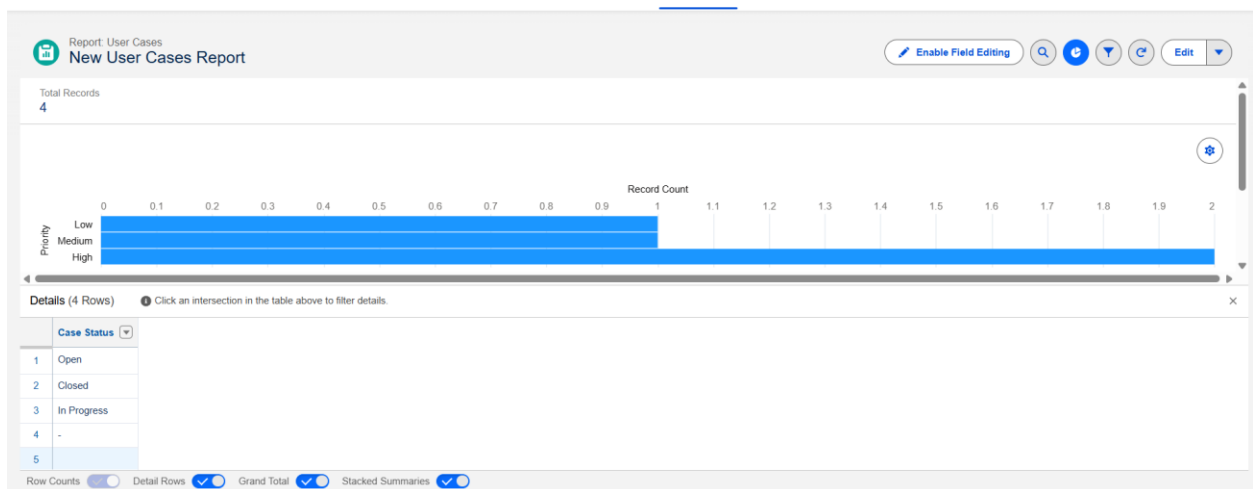
- **Description:** This screenshot captures the **SLA Flow**, specifically the **Scheduled Path** configured to trigger an update or notification after a 3-hour resolution window.

iv. GitHub Repository and Version Control



- **Description:** This provides evidence of the project's source-driven development, showing the **SFDX folder structure** and the recent commits for the metadata.

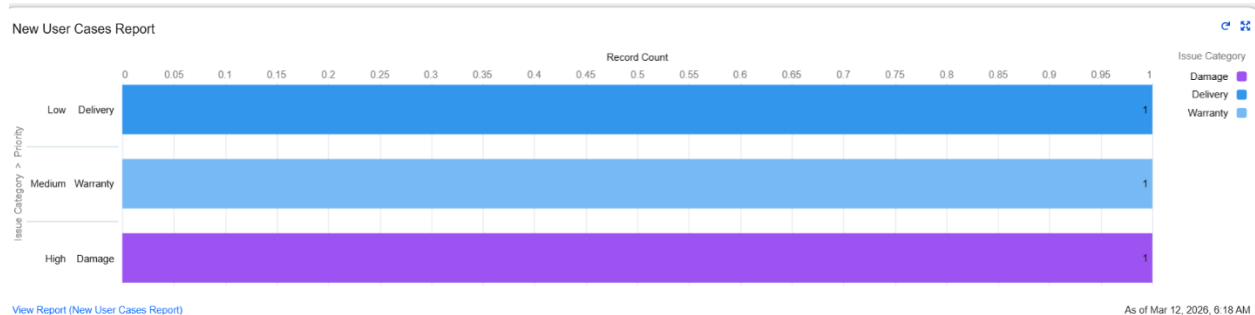
v. Data Analytics and Reporting



- **Title:** New User Cases Detailed Report

- **Description:** This screenshot shows the underlying tabular data and summary chart for the User Case object. It tracks records by Case Status (Open, In Progress, Closed) and Priority. This report allows administrators to drill down into individual records to monitor specific customer issues and overall team progress.

vi. Business Intelligence Visualization(Dashboard)



- **Title:** User Cases Performance Dashboard
- **Description:** This dashboard component provides a visual breakdown of cases by Issue Category (Damage, Delivery, Warranty) cross-referenced with Priority.
 - **Insight:** It highlights that "High Priority" cases are correctly categorized under "Damage," while "Low Priority" cases fall under "Delivery."
 - **Value:** This allows management to see at a glance where the most critical issues are occurring, enabling faster resource allocation.

17. Advantages and Limitations

i. Advantages:-

- **Increased Efficiency:** The Auto-Assignment Flow eliminates manual routing, ensuring that "User Cases" are instantly directed to the correct department based on the "Issue Category."
- **Improved Accuracy: Validation Rules** prevent data entry errors, ensuring that all critical information is captured before a case can be saved.

- **SLA Compliance:** The use of **Scheduled Paths** in Flows ensures that no high-priority case is forgotten, automatically flagging records that exceed resolution deadlines.
- **Enhanced Productivity:** By using **Agentforce** and **Prompt Builder**, support agents receive AI-generated summaries, reducing the time spent reading through long case descriptions.
- **Version Control:** By maintaining the project on **GitHub**, all changes to the system logic are tracked, allowing for easy collaboration and rollback if necessary.
- **Real-time Analytics:** The integration of Reports and Dashboards provides immediate visibility into support trends, allowing the organization to identify recurring product issues (like "Damage" or "Delivery" delays) and address them proactively.

ii. Limitations:-

- **Internet Dependency:** As a cloud-based solution on the Salesforce Lightning Platform, the system is inaccessible without a stable internet connection.
- **Platform Governor Limits:** Salesforce imposes multi-tenant limits (e.g., number of flow interviews or AI prompt calls), which must be monitored as the volume of "User Cases" grows.
- **Licensing Costs:** Advanced features like **Agentforce** and **Prompt Builder** require specific Salesforce licenses, which may be a cost barrier for smaller organizations.
- **Learning Curve:** Users and administrators require specific training to manage the complex interactions between custom objects, flows, and AI grounding.

18. Future Enhancements

- **i. Scalability Improvements:**

- **Standardization of Flow Design:** Transitioning to a "One Flow Per Object" design pattern to ensure that the **User Case** automation remains efficient and easy to manage as more complex rules are added.
- **Collection Filters:** Using advanced collection processing within Flows to handle bulk record updates more effectively without hitting system limits.
- **ii. Feature Expansion:**
 - **Knowledge Integration:** Configuring Salesforce Knowledge to automatically suggest troubleshooting articles to agents based on the "Issue Category" selected on the **User Case**.
 - **Email-to-Case for Custom Objects:** Setting up automated email ingestion so that customer emails are automatically converted into **User Case** records.
- **iii. Cross-Platform Integration:**
 - **Slack for Service:** Using the "Slack for Salesforce" app to push notifications to specific Slack channels whenever a high-priority **User Case** is created or an SLA is breached.
 - **External Services:** Using No-Code "External Services" to connect Salesforce to third-party APIs (like shipping or weather services) to provide more context to the support agent.
- **iv. Mobile Integration:**
 - **Mobile-Specific Layouts:** Designing "Compact Layouts" and "Mobile-Only Actions" for the **User Case** object to allow field agents to update status and upload photos directly from the Salesforce Mobile App.
- **v. Advanced Automation:**
 - **Einstein Next Best Action:** Implementing a declarative recommendation engine that suggests the best next step (e.g.,

"Escalate to Manager" or "Send Refund") to the agent on the record page.

- **Dynamic Forms:** Enhancing the User Case record page with **Dynamic Forms** to hide or show specific fields based on the "Issue Category," keeping the UI clean and user-friendly.

19. Conclusion

i. Project Summary:

The "Salesforce Automated Case Management System" successfully transitions traditional support tasks into a streamlined, automated environment. By utilizing a custom User Case object, the system provides a tailored experience that meets the specific data needs of a modern support department. This project, completed as part of the Experiential Learning training via the Skill Wallet internship, showcases the power of cloud-based CRM solutions.

ii. Technical Implementation:

The implementation highlights a robust "Declarative-First" strategy. Core achievements include:

- **Automated Routing:** Using Record-Triggered Flows to eliminate manual case assignment.
- **SLA Enforcement:** Setting up scheduled paths to ensure "High Priority" cases are addressed within strict 3-hour windows.
- **AI Integration:** Leveraging Agentforce and Prompt Builder to provide intelligent case summarization for faster resolution.

iii. Data-Driven Insights:

As evidenced by the included Reports and Dashboards, the system does not just process records but transforms them into actionable business intelligence. The ability to visualize trends in Issue Categories (Damage, Delivery, Warranty) allows management to make informed decisions regarding resource allocation and process improvements.

iv. Final Impact:

The final solution is a scalable, efficient, and user-friendly application. It reduces administrative overhead, ensures accountability through clear approval processes, and uses version control via GitHub to maintain a professional, source-driven development lifecycle.

20. Appendix

i. Source Code and Version Control

The complete metadata and configuration files for this project are hosted on GitHub. This includes the force-app directory containing the User Case object, Flows, and Approval Processes.

- **GitHub Repository Link:** <https://github.com/Divyanshi9359/salesforce-customer-support-automation.git>

ii. Technical Manifest (package.xml)

The following is a representation of the package.xml used to retrieve and deploy the project metadata via the Salesforce CLI:

XML

```
<?xml version="1.0" encoding="UTF-8"?>

<Package xmlns="http://soap.sforce.com/2006/04/metadata">

  <types>

    <members>User_Case__c</members>

    <name>CustomObject</name>

  </types>

  <types>

    <members>User_Case_Assignment_Flow</members>

    <members>User_Case_SLA_Monitoring</members>

    <name>Flow</name>

  </types>
```

```
<types>
  <members>High_Priority_Approval_Process</members>
  <name>ApprovalProcess</name>
</types>

<version>60.0</version>

</Package>
```

iii. List of Tools and Technologies

- **Salesforce Lightning Platform:** Primary CRM hosting and execution environment.
- **Salesforce Flow Builder:** Core declarative engine for business logic and automation.
- **Salesforce Prompt Builder & Agentforce:** Generative AI tools for record grounding and summarization.
- **Salesforce DX (SFDX):** Command-line interface for source-driven development.
- **Visual Studio Code (VS Code):** Integrated Development Environment (IDE) used for metadata management.
- **GitHub:** Version control system for tracking changes and project collaboration.

iv. Acknowledgments

- **SmartBridge / SmartInternz:** For providing the **Experiential Learning** framework through the Skill Wallet Virtual Internship.
- **Trailhead by Salesforce:** For the learning modules and hands-on developer orgs used during the implementation phase.

